
Per-site retrieval of lunar regolith thermal conductivity K_d at the Apollo 15 & 17 heat-flow boreholes

The physics, the code, and the method — explained from the ground up

A complete companion manuscript to the JGR letter.

R. P. Gregorio • Kasai Laboratory, Institute of Science Tokyo

Abstract

We measure how well heat conducts through the deep lunar regolith — the parameter K_d — separately at the Apollo 15 and Apollo 17 heat-flow boreholes, and find them to differ by about a factor of two ($K_{d,A15}^* = 4.58$, $K_{d,A17}^* = 8.12 \text{ mW m}^{-1} \text{ K}^{-1}$). This document explains *everything* behind that sentence, assuming no prior knowledge: what heat conduction is and how we model it; the two published conductivity models and how to reproduce them; the numerical solver line by line; the subtle bug we found in reaching steady state and the faster, self-certifying method that replaced it; how K_d is actually retrieved from the data; how the uncertainty is quantified; why the basal heat flux Q_b matters; and why the two sites plausibly differ. Figures accompany every step. Sections are cross-referenced to the actual Python source files so you can read this document alongside the code.

How to read this document

Read it straight through. Every chapter builds on the previous one. The document is structured as a single logical argument:

1. What are we measuring and why? (Ch. 1)
2. What physics governs subsurface heat? (Ch. 2)
3. How does conductivity depend on temperature and depth? (Ch. 3)
4. How does the computer solve this? (Ch. 4)
5. Why did the old solver fail, and how does the new one work? (Ch. 5)
6. How do we extract K_d from the data? (Ch. 6)
7. How sure are we? (Ch. 7)
8. What role does Q_b play? (Ch. 8)
9. Why do the sites differ? (Ch. 9)
10. What independent checks confirm the result? (Ch. 10)
11. How is the code organized, and what comes next? (Ch. 11)

Contents

1	What we are trying to do, and why	4
1.1	The question	4
1.2	Why it matters	4
1.3	The data we use	4
1.4	The shape of the whole argument	5
2	The physics, from the ground up	6
2.1	Heat conduction: what Fourier's law says	6
2.2	The heat equation: conservation of energy in a column	6
2.3	The lunar forcing: why the surface oscillates wildly	7
2.4	The skin depth: why only the top matters for the daily cycle	7
2.5	Boundary conditions: what happens at the top and bottom	7
2.5.1	Top (surface): a radiative energy balance	7
2.5.2	Bottom: the geothermal flux	7
2.6	Periodic steady state: the key concept	8
3	The two conductivity models	9
3.1	Hayne et al. (2017): the H-parameter model	9
3.1.1	The formula	9
3.1.2	The published parameters	9
3.1.3	What the code looks like	9
3.1.4	Why the literature K_d is 3.4 — and why we re-measure it	9
3.2	Martínez & Siegler (2021): a density-based model	9
4	The numerical solver, line by line	11
4.1	The geometric depth grid	11
4.2	One time step: Crank–Nicolson	11
4.3	Solving the system: the Thomas algorithm	12
4.4	The non-linear surface: Newton's method	12
4.5	One complete lunation: stepping through the day	12
4.6	The spin-up: repeating lunations until steady state	13
5	Reaching steady state: the old method, the bug, and the fix	14
5.1	Why we need steady state at all	14
5.2	The old method: a fixed 30-lunation spin-up	14
5.3	The bug: what actually went wrong (flag F1)	14
5.4	The fix: the flux-anchored shortcut	15
5.4.1	The key insight	15
5.4.2	Step A: settle the skin and read the anchor	15
5.4.3	Step B: reconstruct the deep profile	15
5.4.4	The small correction: rectified flux	16
5.4.5	Iterating Steps A and B	16
5.5	Proof: they give the same answer	17
5.6	Self-certification: how we know each solve is correct	19

6	Retrieving K_d from the data	20
6.1	One forward run: <code>run_with</code>	20
6.2	The sweep and the minimum	20
6.3	The result	20
7	How sure are we? (uncertainty)	22
7.1	The bootstrap	22
7.2	Bayesian cross-check (MCMC)	23
7.3	Model selection (AICc)	23
8	The role of the basal flux Q_b	24
8.1	The degeneracy: K_d and Q_b trade off	24
8.2	What we do about it	24
8.3	The robust claim	24
9	Why might the two sites differ?	26
10	Independent cross-checks	27
10.1	Diviner surface closure	27
10.2	Martínez α -sweep	27
10.3	Robustness checks	27
11	How the code is organised, and what comes next	29
11.1	Architecture	29
11.2	Parallelism: one solve vs. many solves	29
11.3	C++ port: why and what	29
12	Conclusion: how to defend every claim	31

1. What we are trying to do, and why

1.1 The question

Every airless body loses its internal heat through a blanket of broken rock — *regolith*. How fast that heat escapes, and how steeply temperature rises with depth, is controlled by the regolith’s **thermal conductivity**: a number, K , that says how many watts of heat flow through a square metre for each degree of temperature change per metre of depth (units $\text{W m}^{-1} \text{K}^{-1}$). Deep down, that number settles to a value we call K_d (“ d ” for deep).

The Apollo 15 and 17 missions buried temperature probes in the Moon (the Heat-Flow Experiment, HFE) and recorded subsurface temperatures from 1971 to 1977. The standard practice has been to assume *one* global K_d for the whole Moon. **Our question is simply: what is K_d at each of the two boreholes, separately — and do they differ?** (Fig. 1.1.)

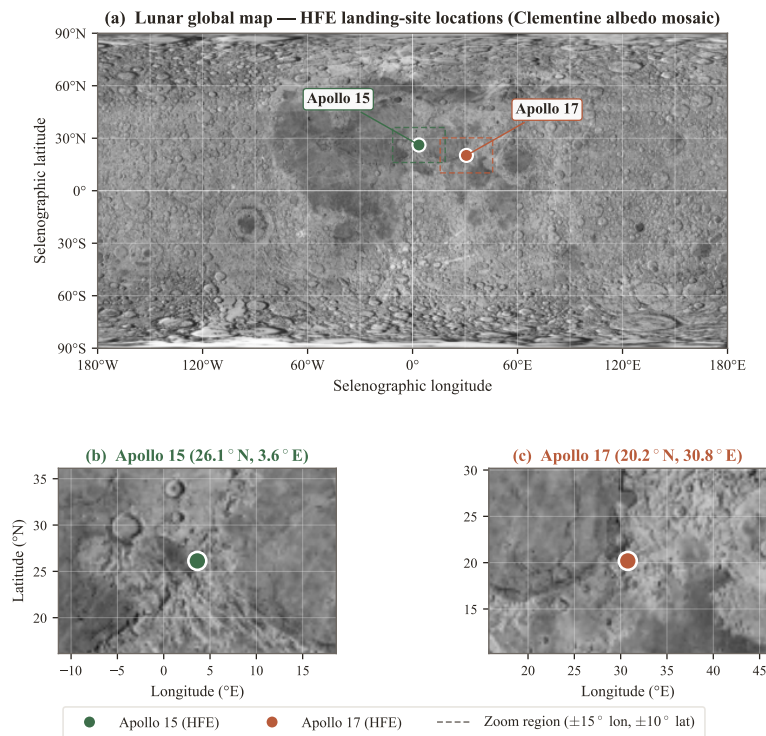


Figure 1.1: The two heat-flow boreholes. Apollo 15 sits at Hadley Rille on the Imbrium mare/Apennine boundary; Apollo 17 in the Taurus–Littrow highland valley. Different terrains suggest possibly different regolith properties.

1.2 Why it matters

K_d sets the deep temperature profile, which feeds models of subsurface ice stability (relevant for future water-mining missions), habitat thermal design, drilling engineering, and the Moon’s heat budget. A wrong, one-size-fits-all K_d propagates errors into all of them. Getting it right *per site* is the contribution of this work.

1.3 The data we use

The Apollo HFE probes recorded temperatures at multiple depths, from about 0.5 m down to 1.39 m (Apollo 15) and 2.34 m (Apollo 17). We use only the *deep sensors* — those below 80 cm — because they sit below the daily temperature swing and respond only to the slow, steady geothermal signal. Their temperatures are stable year to year, which tells us the regolith at those depths has reached its *periodic steady state*: the thermal profile repeats identically from one lunation (lunar day, ~29.5 Earth days) to the next.

At Apollo 15 we have 7 deep sensors; at Apollo 17, 16. These are the data points our model must match.

1.4 The shape of the whole argument

In one breath: (i) heat conduction is described by one equation; (ii) two published formulas give K as a function of temperature and depth; (iii) we put one of them in a computer model of the regolith and run it until it stops changing (“steady state”); (iv) we ask which K_d makes the model’s deep temperatures match the Apollo sensors; (v) the hard part was reaching steady state correctly — the bulk of this document — and (vi) we then quantify the uncertainty and interpret the result. Each clause is a chapter.

2. The physics, from the ground up

2.1 Heat conduction: what Fourier's law says

Heat flows from hot to cold. Fourier's law quantifies this: the heat flux q (watts per square metre, W m^{-2}) through a material is proportional to the temperature gradient:

$$q = -K \frac{\partial T}{\partial z}, \quad (2.1)$$

where z is depth (metres), T is temperature (Kelvin), and K is the thermal conductivity ($\text{W m}^{-1} \text{K}^{-1}$). The minus sign says heat flows *downhill* in temperature: from the hot surface into the cooler subsurface during the day, and back up at night.

In plain words

Imagine holding one end of a metal rod over a flame. Heat flows from the hot end to the cold end. K tells you how fast: copper ($K \approx 400$) carries heat quickly; lunar regolith ($K \approx 0.003$) is an extraordinary insulator. The $\partial T/\partial z$ part says heat flows faster when the temperature changes sharply over a short distance (steep gradient) and slower when it changes gradually (gentle gradient).

2.2 The heat equation: conservation of energy in a column

Now demand energy conservation in a thin slab of regolith. The rate at which the slab stores heat equals the difference between what flows in through its top face and what flows out through its bottom face. This bookkeeping gives the **heat equation**:

$$\rho(z) c_p(T) \frac{\partial T}{\partial t} = \frac{\partial}{\partial z} \left(K(T, z) \frac{\partial T}{\partial z} \right), \quad (2.2)$$

where ρ is the density (kg m^{-3}), c_p is the specific heat capacity ($\text{J kg}^{-1} \text{K}^{-1}$, how much energy it takes to warm a kilogram by one Kelvin), and t is time (seconds).

In plain words

The left side says “this layer is warming (or cooling) at such-and-such a rate.” The right side says “because this much more heat is arriving from above than leaving below (or vice versa).” The three material properties — ρ (how heavy), c_p (how hard to heat), and K (how easily heat passes through) — are the knobs. This single equation is the physics of the entire study.

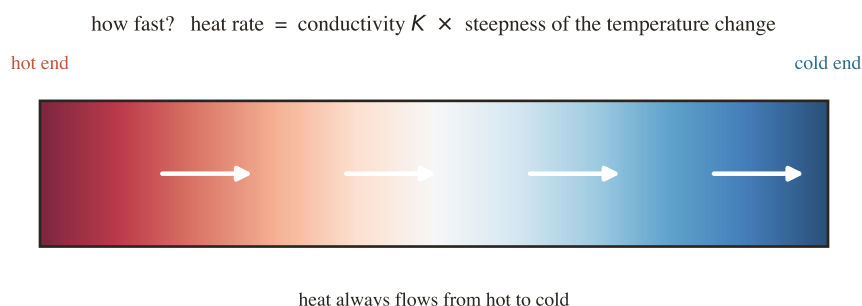


Figure 2.1: Heat conduction in a column: flux $q = -K \partial_z T$ moves heat down the temperature gradient; the heat equation is the bookkeeping of what each layer stores versus what it passes on.

2.3 The lunar forcing: why the surface oscillates wildly

At the surface, the Sun drives a huge day–night temperature swing. The lunar day is about 29.5 Earth days long, so the surface bakes for ~ 14.75 days and then sits in darkness for ~ 14.75 days. Surface temperatures swing from ~ 100 K at night (-173°C) to ~ 390 K at noon ($+117^\circ\text{C}$). This is the “forcing” that drives everything.

2.4 The skin depth: why only the top matters for the daily cycle

This “thermal wave” travels downward, but it *dies out* exponentially. Its amplitude falls by a factor of $e \approx 2.72$ over the **skin depth**:

$$\delta = \sqrt{\frac{2\kappa}{\omega}}, \quad (2.3)$$

where $\kappa = K/(\rho c_p)$ is the thermal diffusivity (how fast heat spreads) and $\omega = 2\pi/P$ is the angular frequency of the forcing (P = one lunation). For lunar regolith, $\delta \approx 10\text{--}20$ cm. Below a few skin depths — roughly 50 cm — the regolith barely feels day or night. It responds only to the *average* heating over the whole lunation.

Key point

This split — a fast, shallow daily wave on top of a slow, deep mean temperature profile — is the single most important fact for everything that follows. The surface oscillates wildly; the deep column sits at a nearly constant temperature that is set by the *mean* energy balance and the upward geothermal flux. Keep this picture in mind through every chapter.

2.5 Boundary conditions: what happens at the top and bottom

The heat equation (Eq. 2.2) needs a rule at each end of the column — these are called **boundary conditions**.

2.5.1 Top (surface): a radiative energy balance

The surface has no atmosphere above it, so sunlight absorbed must equal heat radiated away plus heat conducted downward:

$$(1 - A) S(t) = \varepsilon \sigma T_s^4 + K \left. \frac{\partial T}{\partial z} \right|_{z=0}, \quad (2.4)$$

where:

- A is the albedo (reflectivity, ~ 0.13): fraction of sunlight bounced away;
- $S(t)$ is the incident solar flux (W m^{-2}), which swings between ~ 1361 at noon and 0 at night;
- ε is the emissivity (~ 0.95): how efficiently the surface radiates heat as infrared;
- $\sigma = 5.6704 \times 10^{-8} \text{ W m}^{-2} \text{ K}^{-4}$ is the Stefan–Boltzmann constant;
- T_s is the surface temperature.

The T_s^4 term makes this equation *non-linear* — we cannot solve for T_s by algebra; it must be found numerically each timestep (§4.4).

In plain words

At noon: lots of sunlight arrives, the surface gets very hot, and it radiates that energy away as infrared (T^4 radiation). A tiny bit of heat also conducts down into the soil. At night: no sunlight, so the surface cools by radiating its stored heat, and some heat conducts back up from the still-warm subsurface. The balance between these processes sets T_s at every moment.

2.5.2 Bottom: the geothermal flux

From far below, the Moon’s interior supplies a small, steady heat flux Q_b (“basal flux”). This heat comes from radioactive decay in the Moon’s interior. At Apollo 15, the measured value is $Q_b = 21 \text{ mW m}^{-2}$; at Apollo 17, $Q_b = 15 \text{ mW m}^{-2}$. These are tiny compared to the $\sim 1361 \text{ W m}^{-2}$ noon sunlight, but they set the *deep* temperature gradient.

At the base of our model column, the boundary condition is:

$$K \left. \frac{\partial T}{\partial z} \right|_{z=z_{\max}} = Q_b. \quad (2.5)$$

In plain words

Think of Q_b as a slow trickle of heat rising from below, always on, always the same. It is so weak that at the surface you would never notice it. But at depth — below the daily noise — that trickle is all there is, and it controls how steeply the temperature rises with depth. That is exactly the signal we use to measure K_d .

2.6 Periodic steady state: the key concept

After long enough, the column reaches a **periodic steady state**: every lunation is identical to the last. No net heat is being stored anywhere; the column has fully adapted to both the surface forcing and the basal flux.

Two conditions define this state, and the second one is the backbone of the entire method:

1. **The surface skin repeats:** the shallow daily oscillation is periodic — today’s temperature cycle looks exactly like yesterday’s.
2. **The deep flux is uniform:** below the skin, there are no heat sources, so in steady state the same geothermal flux Q_b must pass upward through *every* layer. Combined with Fourier’s law, this fixes the gradient of the cycle-mean temperature $\langle T \rangle$ at every depth:

$$K(\langle T \rangle, z) \frac{d\langle T \rangle}{dz} = Q_b \quad \implies \quad \frac{d\langle T \rangle}{dz} = \frac{Q_b}{K(\langle T \rangle, z)} \quad (2.6)$$

Key point

Equation (2.6) is the backbone of the whole method. Read it out loud: “the slope of the mean temperature profile at any depth equals the basal flux divided by the conductivity at that depth.” Once you know the temperature at *one* depth, this equation fixes the temperature at every depth below it — because it tells you the slope everywhere. This is not an approximation; it is energy conservation applied to a column in steady state. Chapters 5 and 6 are built entirely on this fact.

In plain words

Analogy: a river flowing through a pipe of varying width. In steady state, the same volume of water passes through every cross-section per second (conservation of mass). Where the pipe is narrow, the water flows fast; where it is wide, the water flows slowly. Similarly: where K is small (poor conductor, like the shallow loose regolith), the temperature gradient must be steep to carry the same Q_b of heat; where K is large (better conductor, like the compacted deep regolith), the gradient is gentle.

3. The two conductivity models

K is not a single number — it depends on temperature (hotter regolith radiates across pores, conducting better) and depth (deeper regolith is more compacted by overlying weight, conducting better). Two published models capture this.

3.1 Hayne et al. (2017): the H-parameter model

3.1.1 The formula

The solid “contact” conductivity rises from a surface value K_s to a deep value K_d as the regolith compacts, over an e -folding depth H :

$$K_c(z) = K_d - (K_d - K_s) e^{-z/H}. \quad (3.1)$$

At the surface ($z = 0$): $K_c = K_d - (K_d - K_s) \cdot 1 = K_s$. At great depth ($z \gg H$): the exponential vanishes, $K_c \rightarrow K_d$. The parameter H controls how fast the transition happens.

A radiative term is added on top — photons can bounce across pore spaces, and this effect grows with T^3 :

$$K(T, z) = K_c(z) \left[1 + \chi (T/T_{\text{ref}})^3 \right]. \quad (3.2)$$

3.1.2 The published parameters

Symbol	Meaning	Value	Source
K_s	surface contact conductivity	0.74 mW m ⁻¹ K ⁻¹	Hayne (2017)
K_d	deep contact conductivity	3.4 mW m ⁻¹ K ⁻¹	Hayne (2017), global
H	compaction e -folding depth	6 cm	Hayne (2017)
χ	radiative enhancement	2.7	Hayne (2017)
T_{ref}	reference temperature	350 K	Hayne (2017)

3.1.3 What the code looks like

In `lunar/properties.py`:

```
def conductivity_hayne(T, z, Ks=7.4e-4, Kd=3.4e-3, H=0.06, chi=2.7):
    Kc = Kd - (Kd - Ks) * np.exp(-z / H) # contact part: K_s → K_d
    return Kc * (1.0 + chi * (T / 350.0) ^3) # + radiative T^3 term
```

3.1.4 Why the literature K_d is 3.4 — and why we re-measure it

Hayne et al. fit Eqs. (3.1)–(3.2) to the *globally averaged* day–night temperature curves measured from orbit by the Diviner instrument. A single set of parameters reproduced the global average, and $K_d = 3.4 \text{ mW m}^{-1} \text{ K}^{-1}$ is that global deep asymptote. It was never intended to be site-specific. Different geological settings — mare basalt versus highland valley debris — might have genuinely different deep conductivities. That is the gap we fill.

In plain words

$K_d = 3.4$ is a whole-Moon average. Our whole project is asking: is the deep regolith at Apollo 15 the same as at Apollo 17? Spoiler: no. One is about twice the other.

The density follows the same exponential shape:

$$\rho(z) = \rho_d - (\rho_d - \rho_s) e^{-z/H}, \quad \rho_s = 1100, \rho_d = 1800 \text{ kg m}^{-3}. \quad (3.3)$$

3.2 Martínez & Siegler (2021): a density-based model

An alternative writes K directly in terms of bulk density ρ :

$$K(T, \rho) = (A_1 \rho + A_2) k_{\text{am}}(T) + (B_1 \rho + B_2) T^3, \quad (3.4)$$

with $k_{\text{am}}(T)$ a fitted amorphous-solid polynomial. Same physics, different parameterisation: where Hayne’s free parameter is K_d , Martínez’s is effectively the deep density. We use it as an independent cross-check (Ch. 10).

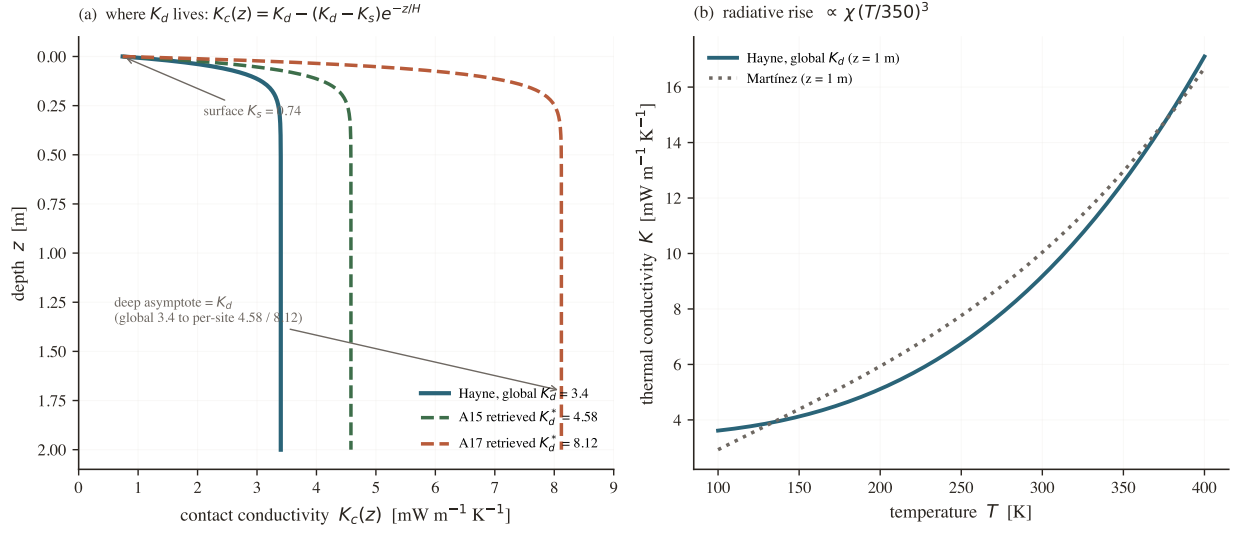


Figure 3.1: (a) Hayne contact conductivity $K_c(z)$: it rises from K_s at the surface to its deep asymptote K_d . The published global value is 3.4; our retrievals move it to 4.58 (A15) and 8.12 (A17). (b) Full $K(T)$ at 1 m for both models — the radiative T^3 term roughly doubles K by midday.

4. The numerical solver, line by line

We cannot solve Eq. (2.2) with pencil and paper, because K depends on T and z in complicated ways. So we **discretise**: chop the column into cells and time into steps, and advance the temperature numerically. This chapter walks through the solver in `lunar/solver.py` and `lunar/grid.py`, piece by piece.

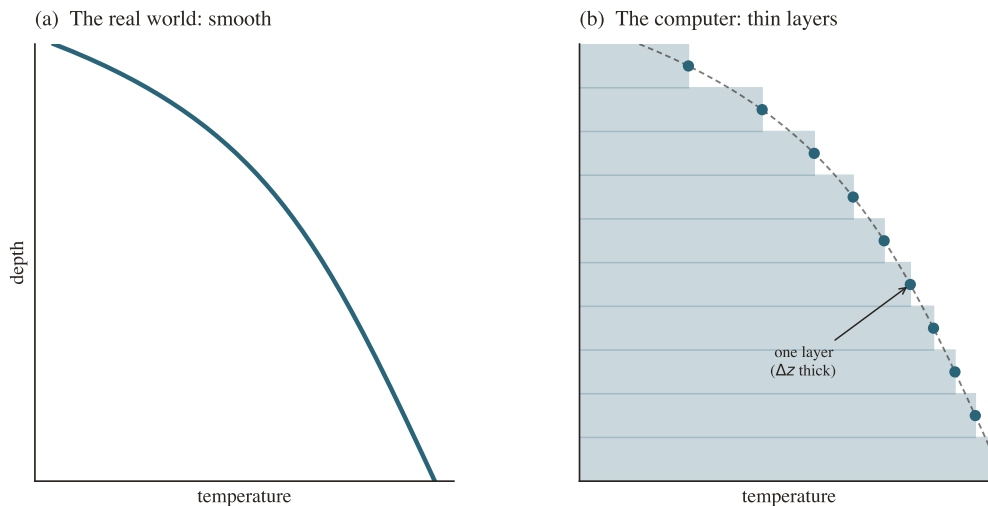


Figure 4.1: Discretising the column: cells in depth, steps in time; each time step solves a small linear system that couples each cell to its neighbours.

4.1 The geometric depth grid

The daily wave needs millimetre resolution at the top (because the temperature changes rapidly over a few centimetres), but the deep column can be coarse (because the temperature there barely changes). So the cells grow geometrically — each $\sim 8\%$ thicker than the one above.

In `lunar/grid.py`:

```
def make_geometric_grid(z_max=5.0, dz0=0.002, growth=0.08):
    faces, dz = [0.0], dz0
    while faces[-1] < z_max:
        faces.append(faces[-1] + dz)
        dz *= (1 + growth) # each layer ~8% thicker
    # cell centres z_mid, thicknesses dz follow from the faces
```

In plain words

The top cell is 2 mm thick (to resolve the daily wave precisely). By a depth of 1 m the cells are ~ 10 cm thick. By 5 m they are ~ 1 m thick. This gives ~ 69 cells total: fine where it matters, cheap where it does not. The alternative — uniform 2 mm cells all the way to 5 m — would need 2500 cells and be $\sim 35\times$ slower.

4.2 One time step: Crank–Nicolson

We step forward in time by one hour (3600 s) per step. There are ~ 709 steps in one lunation. At each step, we need to compute the new temperature profile T^{n+1} from the old one T^n . We use the **Crank–Nicolson** scheme, which averages the heat fluxes at the old and new time levels. This produces a tridiagonal linear system $A T^{n+1} = d$:

In `lunar/solver.py`, function `_step`:

```
K = K_func(T_prev, z_mid) # conductivity at current T, each cell
Kf = harmonic_mean(K) # conductivity at cell faces
cap = rho * cp_func(T_prev) * dz # heat capacity of each cell [J/m^2/K]
```

```

aL = dt * Kf[:-1] / (dz_c[:-1] * cap) # left-face coupling coefficient
aR = dt * Kf[1:] / (dz_c[1:] * cap) # right-face coupling coefficient
# The tridiagonal matrix A has three bands:
a = -0.5*aL # sub-diagonal (coupling to cell above)
c = -0.5*aR # super-diagonal (coupling to cell below)
b = 1 + 0.5*(aL + aR) # main diagonal (self + both neighbours)
# The right-hand side d:
d = 0.5*aL*Tleft + (1 - 0.5*(aL+aR))*T_prev + 0.5*aR*Tright

```

In plain words

What this does, in words: each cell’s new temperature is a weighted average of its old temperature and its neighbours’ temperatures. Cells with high conductivity exchange heat faster (larger coupling coefficients a_L, a_R). Cells with high heat capacity ($\rho c_p \Delta z$) respond more slowly. The 0.5 factors are the Crank–Nicolson averaging between old and new time levels, which makes the scheme second-order accurate and unconditionally stable (no timestep restriction from the grid spacing).

4.3 Solving the system: the Thomas algorithm

A tridiagonal system (each equation involves at most three unknowns) can be solved in just two passes — one forward, one backward — in $O(N)$ time. This is the **Thomas algorithm**:

```

# Forward elimination: sweep top to bottom
for i in range(1, n):
    m = b[i] - a[i]*cp[i-1]
    cp[i] = c[i] / m
    dp[i] = (d[i] - a[i]*dp[i-1]) / m
# Back-substitution: sweep bottom to top
x[n-1] = dp[n-1]
for i in range(n-2, -1, -1):
    x[i] = dp[i] - cp[i]*x[i+1]

```

This is already JIT-compiled with Numba in the code (`_thomas` in `lunar/solver.py`), so it runs at near-C speed.

4.4 The non-linear surface: Newton’s method

The surface energy balance (Eq. 2.4) has a T_s^4 term, so we cannot solve for T_s by simple algebra. We use Newton’s method: guess T_s , compute how far the energy balance is from zero (the “residual” R), compute the slope of R vs. T_s (the “Jacobian” R'), and update:

```

for _ in range(max_iter): # usually converges in 2-3 steps
    R = (1-A)*S - eps*sigma*T_s^4 - Ksurf*(T_s - Tsub)/(0.5*dz0)
    dR = -4*eps*sigma*T_s^3 - 2*Ksurf/dz0 # derivative of R w.r.t. T_s
    T_s = T_s - R/dR # Newton update

```

In plain words

The residual R says: “absorbed sunlight minus radiated heat minus conducted heat — how far from balance are we?” If $R > 0$, the surface is too cold (not radiating enough); if $R < 0$, it is too hot. Newton’s method adjusts T_s in the right direction, overshooting by just the right amount to converge in 2–3 steps.

4.5 One complete lunation: stepping through the day

One lunation = ~ 709 hourly steps. At each step:

1. Compute the solar flux $S(t)$ at the current time.
2. Solve the surface boundary condition by Newton’s method to get T_s .
3. Build the tridiagonal system from the current temperatures.
4. Apply the bottom boundary condition: $K dT/dz = Q_b$ at the base.
5. Solve the tridiagonal system with the Thomas algorithm to get the new temperature profile.
6. Record the temperatures.

After one full lunation we have the complete time-resolved temperature field $T(z, t)$ for that cycle.

4.6 The spin-up: repeating lunations until steady state

To find the steady state, we repeat lunations one after another. After each lunation, we compare the temperature profile to the previous lunation's:

```
for cycle in range(1, n_lunations_spinup + 1):
    T_start = T.copy()
    for k in range(1, n_t): # march through one lunar day
        T = step(T, ...) # the inner time-stepping loop
    if max(abs(T - T_start)) < 0.01 and cycle >= 2:
        break # "converged" -- but is it really?
```

This brute-force approach works *in principle*: given enough lunations, the profile converges to the true steady state. But *how many* lunations is the whole problem of the next chapter.

5. Reaching steady state: the old method, the bug, and the fix

This is the most important chapter. It explains the central methodological contribution of this work: why the original approach to reaching steady state was flawed, what exactly went wrong, and how the new method fixes it. Read this chapter carefully; everything in the retrieval (Ch. 6) depends on it.

5.1 Why we need steady state at all

The Apollo sensors recorded temperatures in regolith that had been sitting undisturbed for billions of years. The thermal profile at depth is in periodic steady state: it repeats exactly every lunation. Our model must produce this same steady state before we can compare it to the data.

K_d is a *steady-state property*: its fingerprint is the settled deep gradient $d\langle T \rangle / dz = Q_b / K_d$ (Eq. 2.6). A higher K_d means heat passes more easily, so the gradient is *gentler* (temperature rises more slowly with depth). A lower K_d means the gradient is *steeper*. Before the model settles, the deep gradient is still forming and reflects the arbitrary starting guess, not K_d .

Warning

You cannot read K_d from an unsettled profile. The deep gradient in an unsettled model is contaminated by the initial condition. It would be like measuring the insulation of a wall while it is still warming up from one side — the temperature slope tells you about the warm-up process, not the insulation.

5.2 The old method: a fixed 30-lunation spin-up

The original code (before this work) solved the problem by running 30 lunations of the time-stepping loop (§4.6). Start from a flat temperature guess (e.g., 250 K everywhere), step through 30 complete lunar days, and declare the result “converged.” The convergence test checked whether the maximum temperature change between consecutive lunations was below 0.01 K.

This seems reasonable: 30 lunations is a month of wall-clock compute. The convergence test passes. But the result was **wrong**.

5.3 The bug: what actually went wrong (flag F1)

The symptom: the retrieved K_d changed when we changed the starting temperature. If we initialised the model at 240 K everywhere, we got one K_d ; at 260 K, we got a different one. But K_d is a property of the regolith, not of our guess — the answer must not depend on where we start.

The diagnosis: the column relaxes to steady state on the *diffusive timescale*:

$$\tau \sim \frac{z_{\max}^2}{\kappa} \approx \frac{(5 \text{ m})^2}{10^{-8} \text{ m}^2/\text{s}} \approx 2.5 \times 10^9 \text{ s} \approx \mathbf{1000} \text{ lunations.} \quad (5.1)$$

After 30 lunations, only ~3% of the relaxation has occurred. The surface skin (top ~50 cm) has settled — it only takes ~10 lunations because the skin is thin. But the deep column (below ~50 cm) still holds the starting guess, essentially untouched.

Why the convergence test lied: the deep drift is so slow — order of 10^{-3} K per lunation — that the cycle-to-cycle change is already below the 0.01 K threshold while the profile is far from settled. The test measures only whether progress has *slowed down*, not whether the profile has actually *arrived*. It’s like measuring the deceleration of a car approaching a traffic light and concluding it has stopped when it’s actually still 100 metres away.

The consequence: the deep sensors at 0.8–2.4 m sit exactly in the unsettled zone. Their modelled temperatures are controlled partly by K_d and partly by the starting guess. When we try to find the K_d that best matches the data, we are fitting a profile that still remembers our arbitrary starting temperature. Different starting temperatures lead to different best-fit K_d values. The starting guess has become a hidden free parameter of the retrieval.

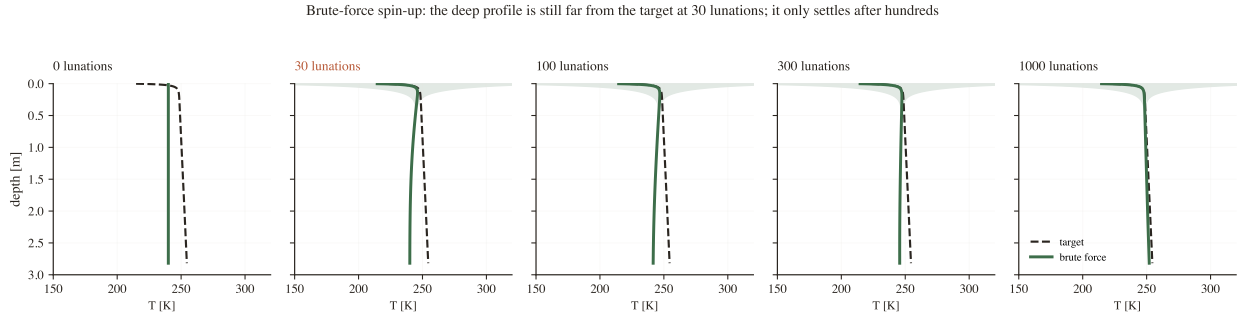


Figure 5.1: The brute-force spin-up shown as snapshots. Green = current profile, dashed = true steady state, shaded = daily swing. At 30 lunations, only the top ~0.5 m has moved; the deep part still sits at the starting guess. The profile only converges after hundreds of lunations. This is why 30 lunations were not enough.

In plain words

Think of it this way: the surface skin equilibrates like a shallow puddle — quickly. The deep column equilibrates like a lake — slowly. Thirty lunations is enough time for the puddle but not the lake. The old code checked whether the puddle had settled, said “yes,” and assumed the lake had too. It hadn’t.

5.4 The fix: the flux-anchored shortcut

The new method avoids waiting for the deep column to relax. Instead of simulating 1000 lunations of heat diffusion, it uses Eq. (2.6) to *calculate* the deep profile directly. This section explains it step by step.

5.4.1 The key insight

Recall the flux closure equation (Eq. 2.6):

$$\frac{d\langle T \rangle}{dz} = \frac{Q_b}{K(\langle T \rangle, z)}.$$

In steady state, the deep temperature slope at *any* depth is determined by Q_b (known) and K at that depth (computable once we know the temperature there). So if we know the temperature at *one* depth below the skin, we can compute the temperature at the next depth down, and the next, and so on — building the entire deep profile without ever simulating the slow diffusive relaxation.

The one temperature we need is the “anchor” — the cycle-mean temperature at a depth just below the daily skin, where the skin has already settled.

5.4.2 Step A: settle the skin and read the anchor

A short spin-up (a handful of lunations) is enough to settle the fast surface skin. After this short run, we read the cycle-mean temperature $\langle T \rangle$ at a chosen **anchor depth** z_0 (0.55 m in the production code, which is about 3 skin depths — well below the daily oscillation zone).

In plain words

The skin only needs ~10 lunations to settle because it’s thin. We don’t need the deep column to settle yet — we only need the temperature at 0.55 m.

5.4.3 Step B: reconstruct the deep profile

This is the step that replaces waiting 1000 lunations. Starting from the anchor temperature at 0.55 m, we “walk” downward through the grid, computing the temperature at each successive depth using the slope rule.

Here is exactly what happens, cell by cell:

1. **Start at the anchor:** we know $\langle T \rangle(z_0)$ from Step A.
2. **Compute the slope:** at this depth, K is known (from the Hayne formula, using the current temperature), and Q_b is a fixed input. So the slope is:

$$\text{slope} = \frac{d\langle T \rangle}{dz} = \frac{Q_b}{K(\langle T \rangle, z_0)}.$$

Example: with $Q_b = 21 \text{ mW m}^{-2}$ and $K \approx 9 \text{ mW m}^{-1} \text{ K}^{-1}$ at depth, the slope is $\sim 2.3 \text{ K/m}$ — temperature increases by about 2.3 K for each metre deeper.

3. **Step to the next cell:** the next cell centre is a distance Δz below. The temperature there is:

$$\langle T \rangle(z_0 + \Delta z) = \langle T \rangle(z_0) + \text{slope} \times \Delta z.$$

4. **Repeat:** move to the new cell, recompute K at the new temperature and depth, get the new slope, step to the next cell. Continue all the way to the bottom of the column.

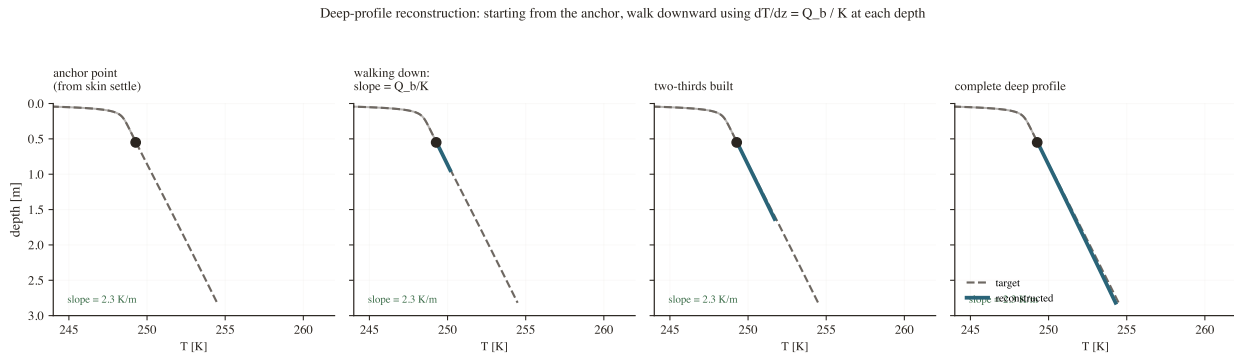


Figure 5.2: The deep profile reconstruction shown as snapshots. Starting from the anchor at 0.55 m (the dot), the code walks downward, computing $dT/dz = Q_b/K$ at each depth to determine the next temperature. By the fourth panel the entire deep profile has been built, matching the true steady state (dashed).

In code (lunar/equilibrium.py, function `_reconstruct_subskin`):

```
T_new = T_mean.copy() # keep the skin as-is from Step A
for i in range(i0, N-1): # walk downward from the anchor
    dz = z[i+1] - z[i] # thickness of this cell
    q = Q_b - u_rect[i] # u_rect = small correction term
    Ki = K(T_new[i], z[i]) # conductivity at this depth and T
    T_half = T_new[i] + 0.5*q/Ki*dz # midpoint temperature (for accuracy)
    K_half = K(T_half, 0.5*(z[i]+z[i+1])) # midpoint conductivity
    T_new[i+1] = T_new[i] + q/K_half*dz # THE STEP: T_next = T_here + slope*dz
```

In plain words

This is the part that people find hardest to believe: how can we know the deep temperature *without simulating* it? The answer is that in steady state, energy conservation fixes the answer. The same Q_b watts flows through every layer. At each layer, Fourier’s law tells us what temperature gradient is needed to carry that flux through that layer’s conductivity. So we just walk downward, adding “slope $\times$ step” at each cell. The deep profile draws itself.

It is exactly like calculating the slope of a hillside: if you know the elevation at one point and the slope at every point (from a map), you can compute the elevation at any other point by adding up slope $\times$ distance.

5.4.4 The small correction: rectified flux

There is one subtlety. Near the skin/deep boundary, the daily temperature oscillation is small but not zero. The oscillation creates a tiny “eddy” heat flux (called the **rectified flux**, u_{rect}) that slightly modifies the mean heat transport. The code subtracts this correction: $q = Q_b - u_{\text{rect}}$. Below about 1 m, the oscillation is negligible and $u_{\text{rect}} \approx 0$, so the slope is simply Q_b/K .

5.4.5 Iterating Steps A and B

Steps A and B are repeated a few times. Why? Because:

- Step A settles the skin assuming the deep column is as we left it.
- Step B reconstructs the deep column using the anchor from Step A.
- With the new deep column, Step A produces a slightly different anchor (because the boundary between skin and deep has shifted a bit).

- Step B then gives a slightly different deep profile.

But each round the changes are smaller. After 3–5 rounds, the anchor temperature moves by less than 0.005 K between iterations — we declare convergence.

In code (`lunar/equilibrium.py`, function `solve_periodic_equilibrium`):

```
stages = [
    dict(z0=0.25, n_in=4, max_it=4, tol=0.10), # coarse, fast
    dict(z0=0.55, n_in=12, max_it=20, tol=0.005), # fine, accurate
]
T_init = np.full(N, T_guess) # arbitrary starting guess

for stage in stages:
    anchor = find_cell_nearest(z, stage.z0)
    prev = infinity
    for it in range(stage.max_iters):
        out = solve_pixel(T_init, n_lunations=stage.n_in) # STEP A: settle skin
        T_mean = out.T.mean(axis=1) # cycle-mean profile
        u = rectified_flux(out.T, z, K) # eddy correction
        T_recon = reconstruct(T_mean, anchor, Q_b, K, u) # STEP B: deep profile
        drift = abs(T_mean[anchor] - prev)
        if drift < stage.tol and it >= 2:
            break # converged!
        prev = T_mean[anchor]
        T_init = T_recon # feed back into next Step A
```

The two-stage schedule (first anchor at 0.25 m, then at 0.55 m) is an optimisation: the shallow anchor converges fast and gets the profile into the right ballpark; the deeper anchor then refines it to high accuracy.

The new method reaches the SAME target in a few steps — the ‘reconstruct’ move snaps the deep profile straight onto the steady gradient

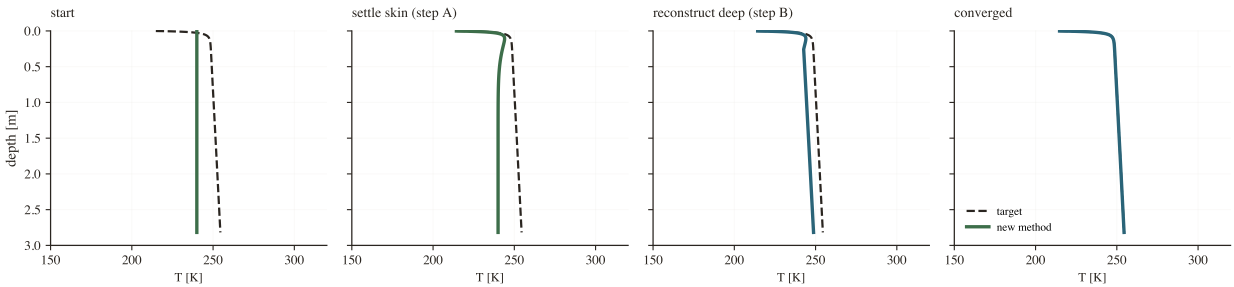


Figure 5.3: The new method. Start from a deliberately wrong flat guess (240 K). Step A settles the skin; Step B’s reconstruction snaps the deep profile onto the steady gradient. After a few rounds, the profile matches the true steady state (dashed). Total cost: ~150 lunations of stepping (~9 s), versus ~1000+ lunations (~4 min) for brute force.

5.5 Proof: they give the same answer

The decisive test has two parts.

Part 1 — the old method is guess-dependent. We ran the old brute-force method (30 lunations) from two starting guesses: 250 K and 200 K. The two resulting profiles *disagree* at sensor depths by ~40 K (Fig. 5.5a). The answer you get depends on where you start — a sign that 30 lunations is nowhere near enough.

Part 2 — the new method is guess-independent. We ran the new flux-anchored method from three wildly different starting temperatures: 200 K, 250 K, and 300 K. All three produce the *same* profile to within 21 mK at sensor depths (Fig. 5.5b,c). Different starting points, same destination.

Why does the old method need so long? Fig. 5.4 shows brute-force error at sensor depths falling as a power law with lunation count; at 30 lunations, the error is still ~13 K. The deep column’s thermal timescale is $\tau \sim 10^3$ lunations — the old method was waiting for heat to diffuse; the new method imposes the answer directly via $dT/dz = Q_b/K$.

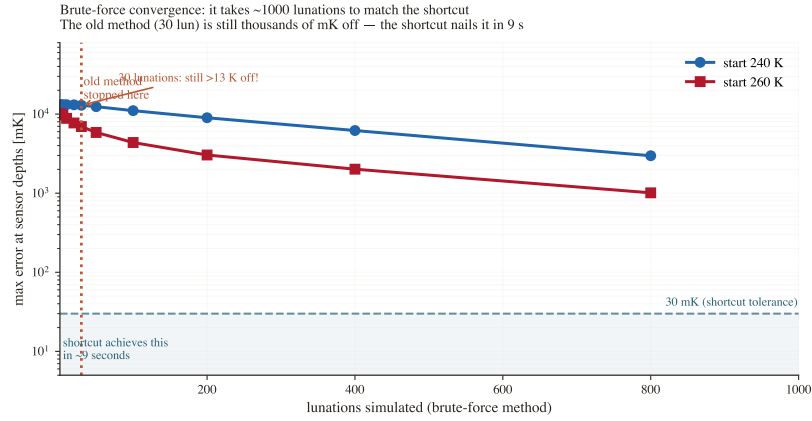


Figure 5.4: Brute-force error at sensor depths (0.8–2.4 m) vs. lunation count. At 30 lunations (red dashed line) the old method still carries >12 K of error. The shaded band marks the <30 mK tolerance that the shortcut achieves in ~9 s.

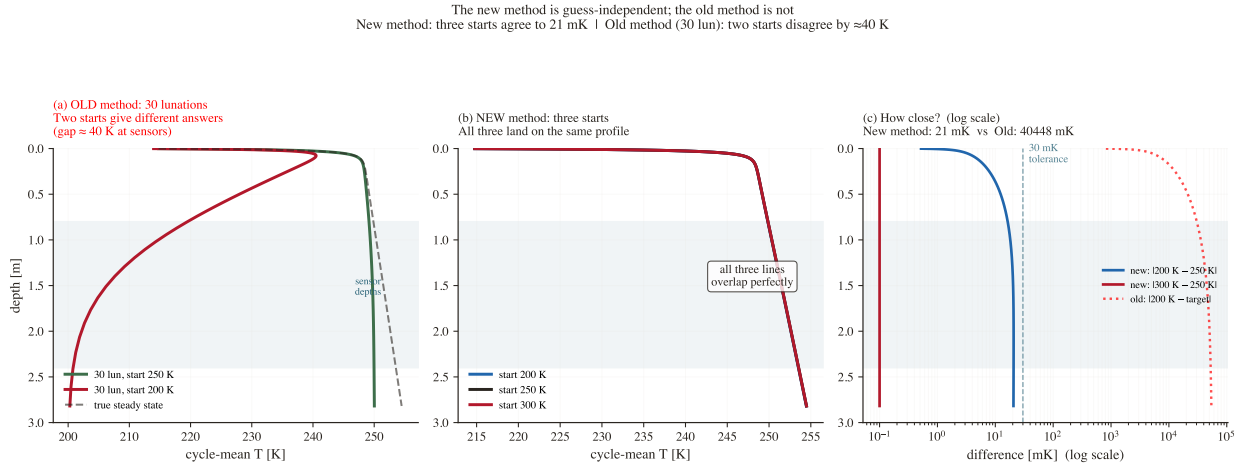


Figure 5.5: Guess-dependence test. (a) The old method (30 lunations) from two starting guesses gives different answers — a ~40 K gap at sensor depths. (b) The new method from three starting guesses (200, 250, 300 K) lands on the same profile. (c) Residual: the new method’s three runs agree to <21 mK at sensor depths; the old method’s error exceeds 12 K. **This figure is the proof that the new method converges to a unique, guess-independent steady state.**

5.6 Self-certification: how we know each solve is correct

Every call to `solve_periodic_equilibrium` returns two diagnostic numbers:

1. **Anchor drift:** how much the anchor temperature changed between the last two iterations. Must be < 0.03 K; typically 3 mK (0.003 K). This proves the iteration has converged.
2. **Flux closure:** the maximum relative deviation of the mean flux from Q_b at all depths below the anchor: $\max |\langle q \rangle(z) - Q_b| / Q_b$. Must be $< 2\%$. This proves the profile satisfies energy conservation.

If either check fails, the code signals that the solve did not converge. This is exactly the verification the old fixed spin-up could never provide.

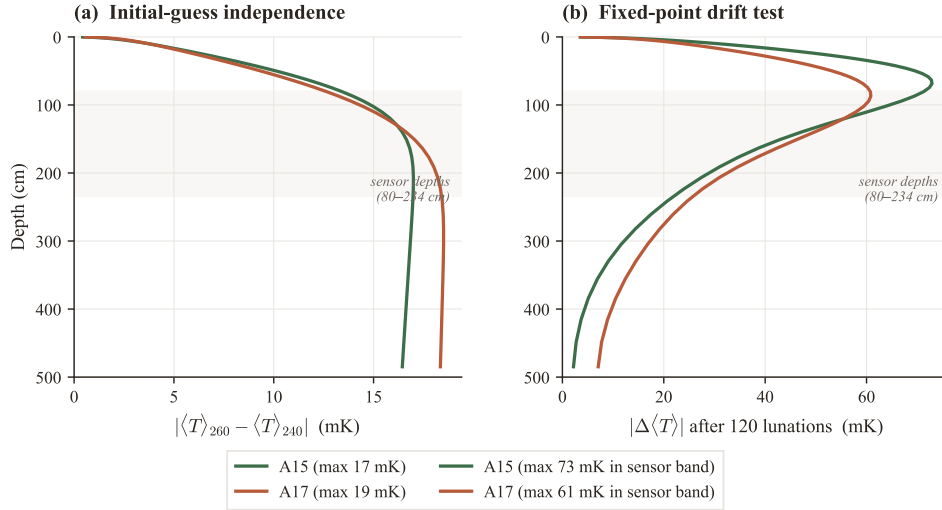


Figure 5.6: Guess-independence and flux closure — the returned proof that a solve reached steady state. Different starting guesses produce the same answer (left); the flux closes on Q_b to $< 2\%$ at all depths (right).

6. Retrieving K_d from the data

Now we have a solver that produces a correct, certified steady-state profile for any trial value of K_d . The retrieval is conceptually simple: try many K_d values, compare each to the data, and pick the best one.

6.1 One forward run: `run_with`

The hub of the pipeline turns one trial K_d into one settled profile. This one function connects the grid, the conductivity model, the solver, and the equilibrium driver:

```
def run_with(site, kd):
    grid = make_geometric_grid(^GRID) # 69-cell geometric grid
    t = np.arange(0, T_LUNAR, DT_STEP) # one lunation of hourly steps
    insol = (1 - site.albedo) * S0 * cos(...) # sunlight forcing
    K = lambda T, z: conductivity_hayne(T, z, Kd=kd) # Hayne with this K_d
    eq = solve_periodic_equilibrium(
        grid, t, insol,
        Q_b=site.Q_b, K_func=K, ...) # →certified steady state
    return grid.z_mid, eq.T_mean # depths and settled mean profile
```

Each call to `run_with` takes about 9 seconds (the new equilibrium solver). It returns the mean temperature at every depth in the grid.

6.2 The sweep and the minimum

We call `run_with` for a grid of K_d values (28 values for Apollo 15, spanning 1–15 mW m⁻¹ K⁻¹; 30 for Apollo 17, spanning 3–25). For each K_d , we measure the misfit between the model’s deep temperatures and the Apollo sensor measurements:

$$\text{RMSE}(K_d) = \sqrt{\frac{1}{N} \sum_{i=1}^N [\langle T \rangle(z_i; K_d) - T_i^{\text{obs}}]^2}, \quad (6.1)$$

where the sum runs over the N deep sensors ($N = 7$ for A15, $N = 16$ for A17).

The best-fit K_d^* is the value that minimises this RMSE. We refine the minimum with a parabolic fit through the three points around the grid minimum:

```
rmse = np.sqrt((residuals^2).mean(axis=0)) # RMSE at each trial K_d
k = np.argmin(rmse) # nearest grid minimum
kd_star = parabola_vertex(kd_grid[k-1:k+2], # sub-grid refinement
                           rmse[k-1:k+2])
```

Each site shows a clean single minimum (Fig. 6.1). The curves are smooth because the solver is deterministic and the steady state is unique for each K_d .

6.3 The result

$$K_{d,A15}^* = 4.58, \quad K_{d,A17}^* = 8.12 \quad \text{mW m}^{-1} \text{K}^{-1}.$$

Apollo 17’s deep regolith conducts heat about twice as well as Apollo 15’s. Both are higher than the global literature value of 3.4.

With these retrieved values, the modelled deep temperatures track the data far better than the one-size-fits-all value (Fig. 6.2); at Apollo 17 the misfit roughly halves.

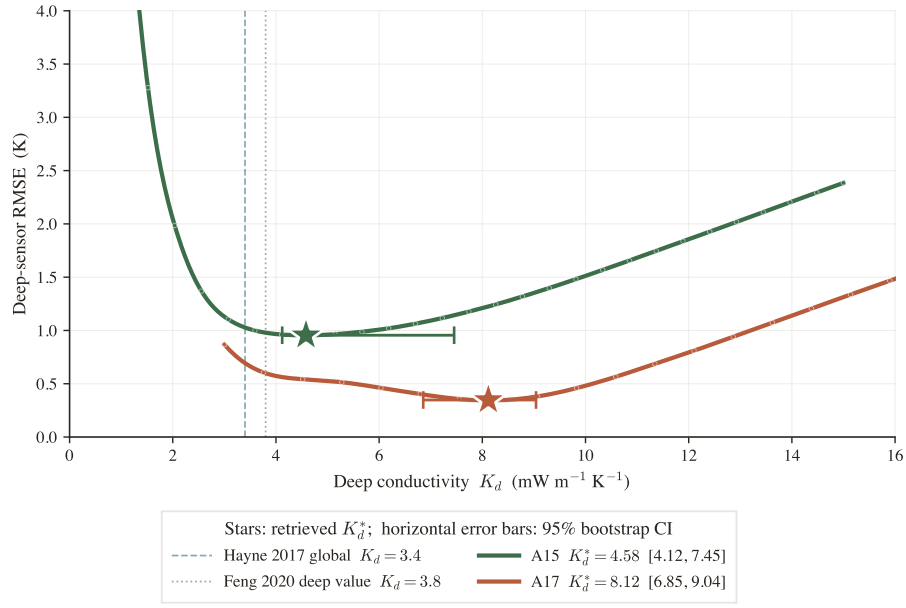


Figure 6.1: Deep-sensor RMSE vs. trial K_d . The minimum is K_d^* ; Apollo 17's sits at a clearly higher conductivity than Apollo 15's.

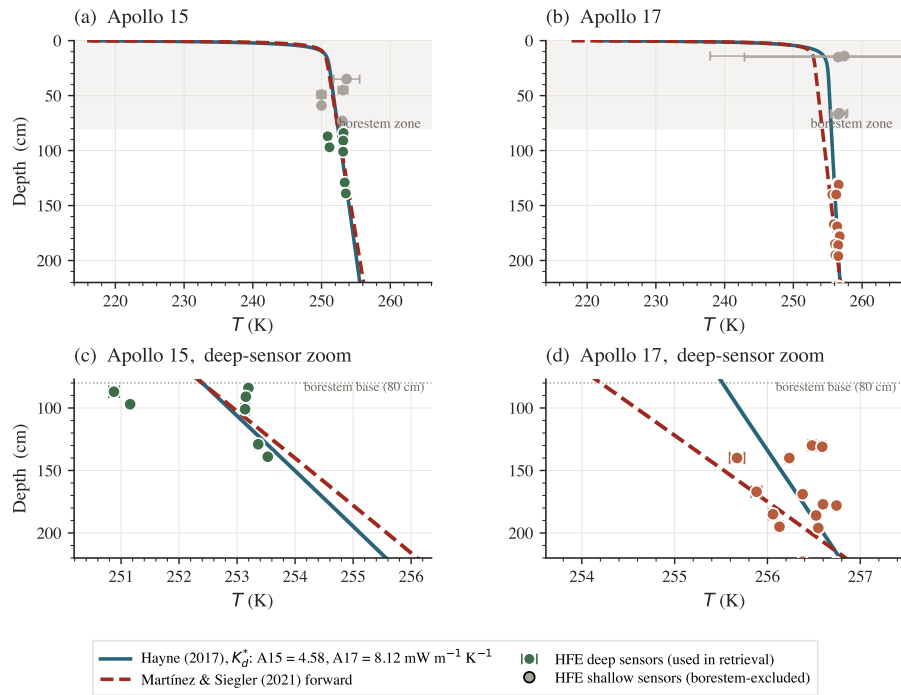


Figure 6.2: Settled mean temperature vs. depth using each site's retrieved K_d^* , plotted against the deep-sensor measurements (dots). The per-site K_d matches the data; the global $K_d = 3.4$ does not.

7. How sure are we? (uncertainty)

A point estimate ($K_d^* = 4.58$) is not useful without knowing how uncertain it is. We quantify uncertainty three ways.

7.1 The bootstrap

The bootstrap is a resampling technique that does not assume any particular noise model. The procedure:

1. **Resample the sensors:** draw N sensors from the original N *with replacement* (some sensors appear twice, others not at all). Also jitter each sensor's depth by a random amount drawn from $\mathcal{N}(0, 2.5 \text{ cm})$ to account for the uncertainty in exactly where each sensor is positioned (Nagihara 2018).
2. **Re-fit K_d^* :** run the full sweep on this resampled sensor set.
3. **Repeat $N_{\text{boot}} = 1500$ times.**
4. **Read the spread:** the 2.5th and 97.5th percentiles of the 1500 K_d^* values give the 95% confidence interval.

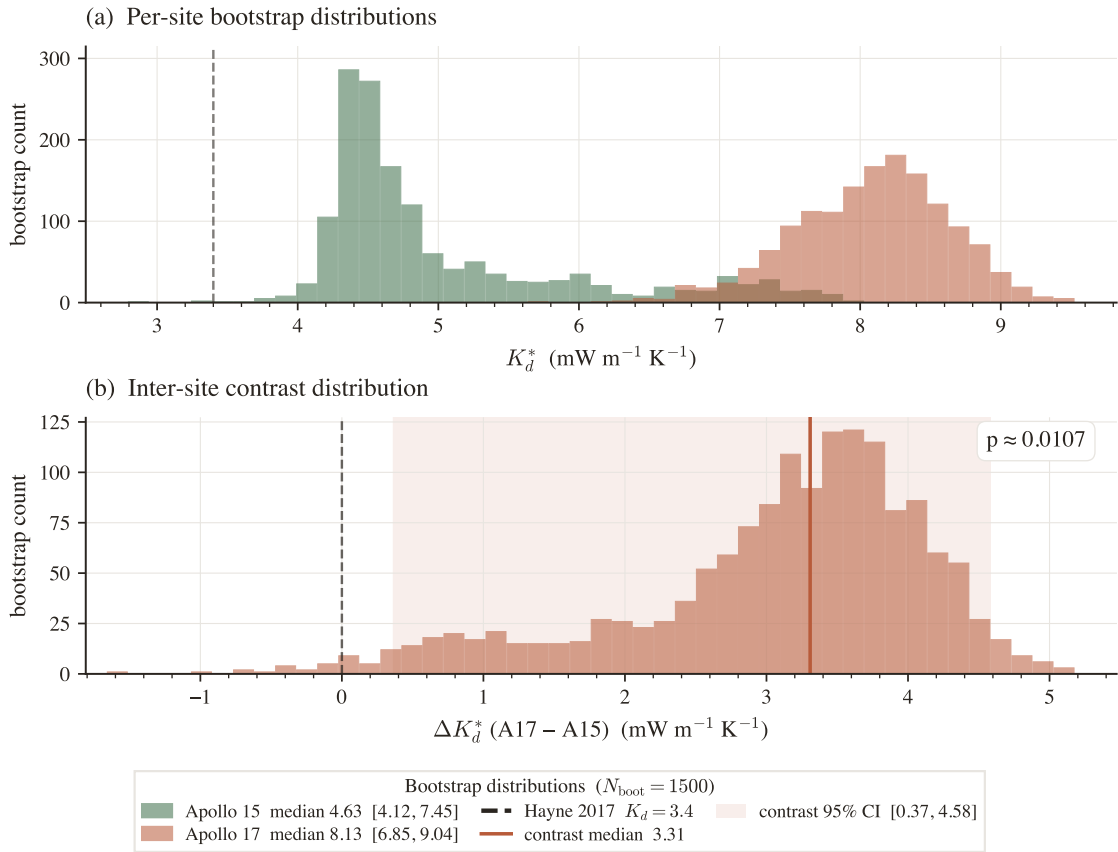


Figure 7.1: Bootstrap distributions of K_d^* (1500 resamples). The two site distributions barely overlap — the contrast is statistically real ($p \approx 0.011$).

The results:

	K_d^* (best fit)	95% CI	
Apollo 15	4.58	[4.12, 7.45]	$\text{mW m}^{-1} \text{K}^{-1}$
Apollo 17	8.12	[6.85, 9.04]	$\text{mW m}^{-1} \text{K}^{-1}$
Contrast (A17 – A15)	3.31	[0.37, 4.58]	$p \approx 0.011$

The p -value of ~ 0.011 means: the probability of seeing this large a contrast (or larger) by chance, if the two sites actually had the same K_d , is about 1%. The two humps in Fig. 7.1 barely overlap.

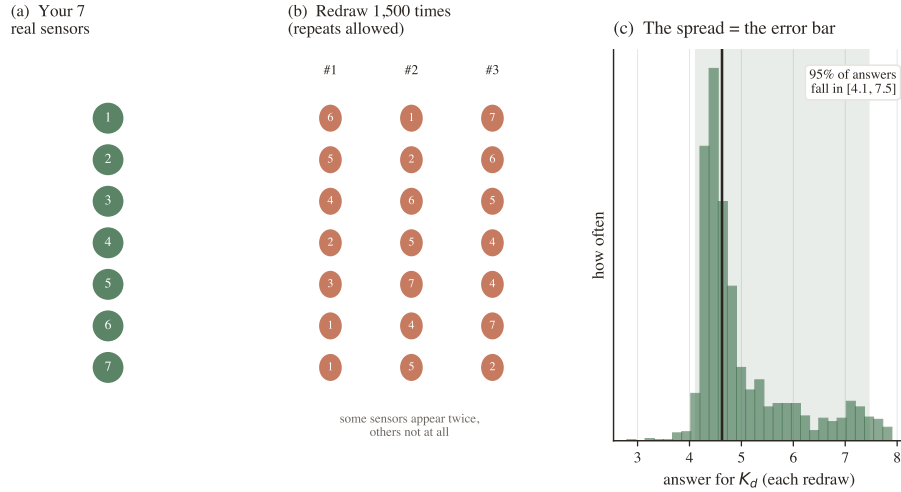


Figure 7.2: What the bootstrap does, schematically: resample the data many times, re-fit each time, read the spread.

7.2 Bayesian cross-check (MCMC)

An independent check: instead of sweeping K_d on a grid, we run a Markov-chain Monte Carlo (MCMC) sampler that explores the full joint posterior in (K_d, Q_b) — allowing both parameters to vary, not just K_d . Even when Q_b is uncertain, the posterior shows Apollo 17 is more conductive than Apollo 15 about 83% of the time.

7.3 Model selection (AICc)

We compare three models:

- M_1 : different K_d per site, Q_b fixed (our model, 2 free parameters);
- M_2 : one shared K_d , Q_b fixed (1 free parameter);
- M_3 : one shared K_d , different Q_b per site (3 free parameters).

AICc (Akaike Information Criterion, corrected for small samples) penalises models for using more parameters. M_1 wins: $\Delta\text{AICc}(M_2 - M_1) = 3.99$, $\Delta\text{AICc}(M_3 - M_1) = 2.71$. The data favour a *conductivity* contrast, not a flux artefact.

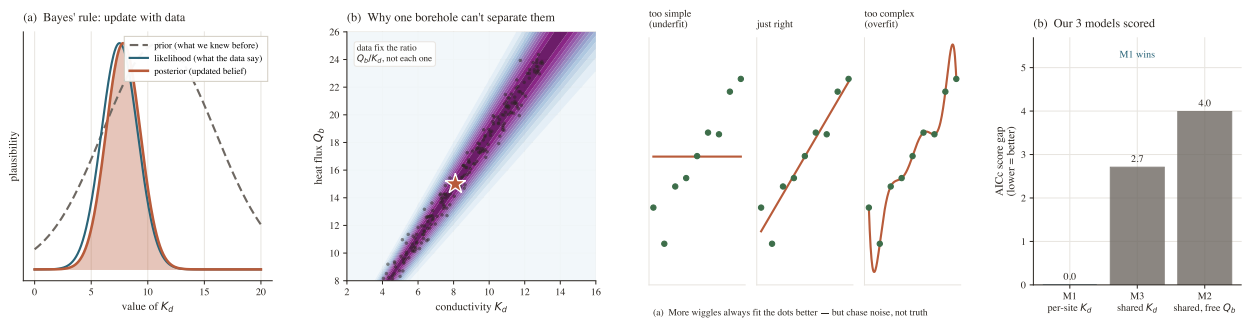


Figure 7.3: Left: the idea of MCMC posterior sampling. Right: AICc model selection trades fit quality against the number of free parameters.

8. The role of the basal flux Q_b

K_d cannot be read in isolation, and understanding *why* is essential.

8.1 The degeneracy: K_d and Q_b trade off

The deep gradient is Q_b/K (Eq. 2.6). For the *same* set of deep temperatures, you could explain the observed gradient with:

- high Q_b + high K_d (strong flux, easily conducted), or
- low Q_b + low K_d (weak flux, poorly conducted).

Many combinations along a diagonal in (K_d, Q_b) space fit the data almost equally well. This is the K_d – Q_b **degeneracy**. We cannot separate the two from a single borehole’s temperature measurements alone.

8.2 What we do about it

We *fix* Q_b at the published values (21 mW m^{−2} for A15, 15 for A17) and report K_d *conditional* on them. This is honest: we are not claiming to have measured K_d independently of Q_b . The headline number $K_d^* = 4.58$ means “the K_d that best fits the data *given* $Q_b = 21$.”

8.3 The robust claim

Even though the absolute K_d values shift when Q_b changes, the *direction* of the contrast survives: Apollo 17 remains more conductive than Apollo 15 for all plausible Q_b values (~83% posterior probability from the MCMC). Q_b is one of the largest terms in the error budget — a direct fingerprint of this coupling.

In plain words

If someone challenges your K_d by saying “but what if Q_b is different?” — acknowledge it: “yes, K_d rides on Q_b . If Apollo 15’s Q_b were higher, our K_d would be higher too. The robust finding is not the absolute number but the direction: Apollo 17 is more conductive than Apollo 15. That holds regardless of reasonable Q_b assumptions.”

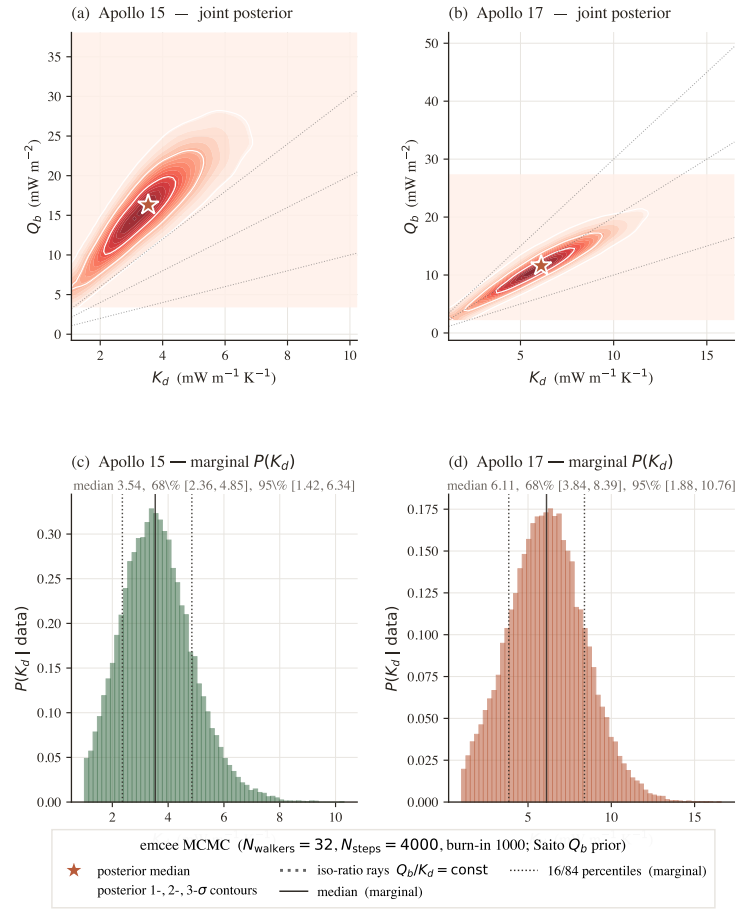


Figure 8.1: The K_d – Q_b trade-off: many combinations along a diagonal fit almost equally well, so K_d is conditional on Q_b . The contrast between sites is more robust than either absolute value.

9. Why might the two sites differ?

A factor-of-two contrast is large but physically reasonable: the two boreholes sample genuinely different regolith.

	Apollo 15	Apollo 17
Setting	Hadley Rille (mare/front)	Taurus–Littrow valley
Bond albedo	0.131	0.137
Deepest sensor	1.39 m	2.34 m
Basal flux Q_b (fixed)	21 mW m ⁻²	15 mW m ⁻²
Retrieved K_d^*	4.58	8.12 mW m⁻¹ K⁻¹

Heat moves through dry regolith mainly by grain-to-grain contact, so a higher K_d points to *denser, more compacted, or more rock-rich* deep regolith. Three plausible contributors at Apollo 17:

1. **Material:** highland massif debris plus dense pyroclastic dark-mantle deposits, versus Hadley’s mare-basalt regolith.
2. **Independent support:** the density-based Martínez model prefers a higher effective deep density at Apollo 17 (Ch. 10).
3. **Compaction:** more buried rock fragments or a more mature, sintered column at Taurus–Littrow.

The honest framing: *Apollo 17’s deep regolith conducts heat about twice as efficiently as Apollo 15’s, conditional on the published basal fluxes.*

10. Independent cross-checks

We test the result against two independent data sources.

10.1 Diviner surface closure

Run the model forward with the retrieved K_d and compare its predicted *surface* day–night curve to orbital Diviner data. This is a check that uses *no heat-flow data* — it only asks whether the deep-sensor-derived K_d also produces a reasonable surface signature. They agree (Fig. 10.1), so the deep fit and the surface are self-consistent.

10.2 Martínez α -sweep

Repeat the retrieval with the entirely different density-based conductivity model (Eq. 3.4). Each site prefers a different effective deep density — telling the same story of inter-site contrast through an independent set of physical assumptions (Fig. 10.1).

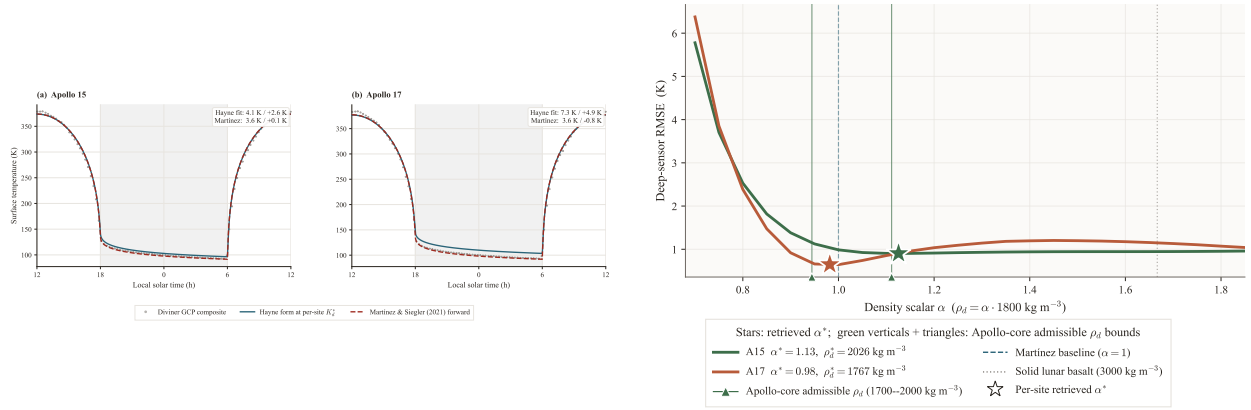


Figure 10.1: Left: the model with the retrieved K_d reproduces the Diviner surface temperatures. Right: the Martínez density retrieval tells the same story — different deep densities at the two sites.

10.3 Robustness checks

Additional sensitivity tests (Fig. 10.2):

- **Joint (K_d, H) surface:** simultaneously varying K_d and the compaction scale height H shows a single, well-defined minimum.
- **Input sensitivities:** perturbing albedo, emissivity, Q_b , and H individually and re-fitting K_d quantifies how each input affects the answer.

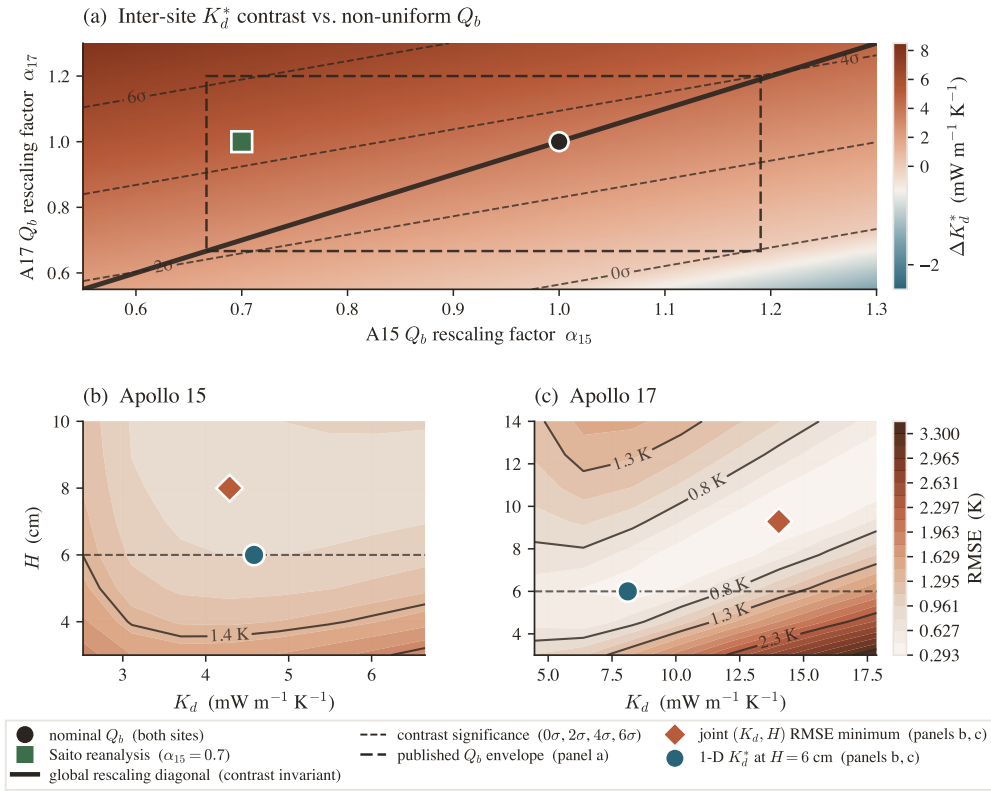


Figure 10.2: Robustness: (a) joint K_d – H RMSE surface, (b,c) sensitivity of K_d^* to input perturbations.

11. How the code is organised, and what comes next

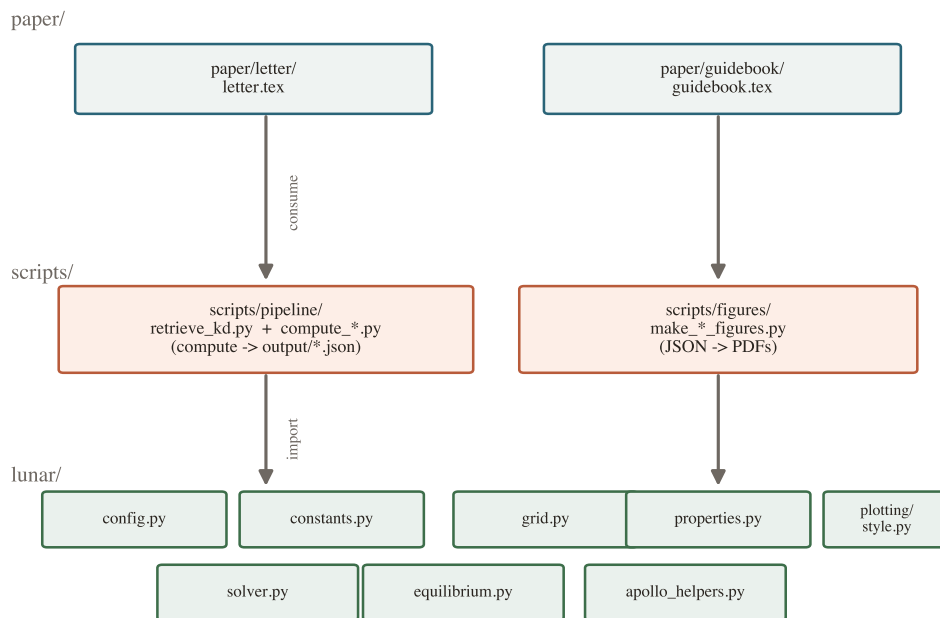
11.1 Architecture

The code is layered (Fig. 11.1):

- **lunar/** — the physics package. Contains the models (`properties.py`), the depth grid (`grid.py`), the time-stepping solver (`solver.py`), and the equilibrium driver (`equilibrium.py`). All constants live in `constants.py`; all run configuration in `config.py`.
- **scripts/** — thin drivers that call `run_with` and produce figures or output files.
- **notebooks/** — interactive exploration (the equivalence demo, the retrieval, the figures).
- **tests/** — 37 tests including equilibrium certification.

One function, `run_with` (in `scripts/pipeline/run_retrieval.py`), is the hub: it wires together the grid, the conductivity model, and the solver, and returns a settled profile. Everything else — the sweep, the bootstrap, the cross-checks — calls `run_with` in a loop.

The three-layer architecture



Dependencies point only DOWNWARD — nothing in `lunar/` imports a script.

Figure 11.1: Three layers; imports point only downward. The numerics live in three small modules (`grid`, `solver`, `equilibrium`).

11.2 Parallelism: one solve vs. many solves

A single forward solve (one call to `run_with`) is inherently sequential: each time step depends on the previous one. You cannot split a single trajectory across cores.

But the *sweep* (28–30 independent K_d values) and the *bootstrap* (1500 independent resamples) consist of many independent solves. These run across all available cores. “More cores speed up the batch, not one solve.”

11.3 C++ port: why and what

The inner time step (`_step` in `solver.py`) runs ~709 times per lunation and ~150 lunations per equilibrium solve — over 100,000 calls. The Thomas algorithm is already JIT-compiled (Numba), but the matrix assembly is interpreted Python — exactly where a compiled language wins (~10–100×).

The plan: port grid, solver, and equilibrium to C++; keep configuration and analysis in Python. Validate the C++ port against the existing 37-test suite and the committed steady-state profiles.

The three speed-ups multiply:

$$\underbrace{\text{algorithm (shortcut)}}_{\sim 30\times, \text{ done}} \times \underbrace{\text{compilation (C++)}}_{\sim 10\text{--}100\times} \times \underbrace{\text{parallelism}}_{\times \text{cores}}.$$

Not needed for this paper's ~ 60 solves; valuable at scale (full Diviner maps, mission pipeline).

12. Conclusion: how to defend every claim

- **What we did.** Retrieved the deep regolith conductivity separately at Apollo 15 and 17: 4.58 and 8.12 $\text{mW m}^{-1} \text{K}^{-1}$.
- **How.** A 1-D heat model (Ch. 2–4) with the Hayne conductivity (Ch. 3), solved to a certified steady state (Ch. 5), with K_d chosen to minimise the deep-sensor misfit (Ch. 6).
- **Why the solver is correct.** It is unique by construction, certifies its own convergence, is guess-independent, and is confirmed by brute force converging onto it (§5.5).
- **Why the result is real.** The contrast is statistically significant (bootstrap $p \approx 0.011$, Ch. 7), survives Q_b uncertainty (83% posterior, Ch. 8), and is echoed by an independent model (Ch. 10).
- **The honest caveat.** K_d is conditional on the assumed Q_b (Ch. 8); the robust statement is the inter-site *direction*.
- **The physics.** Apollo 17’s deep regolith conducts heat about twice as efficiently — plausibly denser, more compacted, or more rock-rich material (Ch. 9).

Reproduce everything: `make retrieve` (the result), `notebooks/equilibrium_demo.ipynb` (old vs. new), and `pytest tests/test_equilibrium.py` (the certification).